

KNOWLEDGE BYTES FOR DOCKER IMPLEMENTATION

Byte 1: What Docker Is Doing

Builds on:

None — starting point

In plain terms:

Docker packages an application and its required software into a **container**. A container provides an isolated environment in which the application can run.

In our project, instead of running Vue, ASP.NET Core, PostgreSQL, and NGINX separately on Windows, they are packaged together into **one Docker container**.

The code:

```
docker compose up -d
```

What's happening:

Docker Compose reads docker-compose.yml and starts the single app container.

Why it matters:

We can start the entire Dockerized Placement Tracker with one command.

Byte 2: Why WSL 2 Is Used on Windows

Builds on:

Byte 1

In plain terms:

Docker needs a Linux environment to run our Linux-based container. Since the host machine is Windows, Docker Desktop uses **WSL 2** to provide the Linux environment.

The code:

```
wsl --status
```

```
wsl --list --verbose
```

What's happening:

These commands show the WSL configuration and distributions available on the computer.

Why it matters:

WSL 2 allows Docker Desktop to run Linux containers while we continue using Windows.

Byte 3: Docker Desktop Provides the Linux Environment

Builds on:

Byte 2

In plain terms:

We are not manually running the Docker project inside Ubuntu.

Docker Desktop provides its own WSL 2 distribution called:

docker-desktop

The architecture is:

Windows



Docker Desktop



docker-desktop (WSL 2)



Docker Engine



Our single Linux container

What's happening:

Docker Desktop manages the Linux environment required by Docker.

Our separate Ubuntu WSL distribution is **not responsible for running this project**.

Why it matters:

This explains why we can execute Docker commands directly from Windows PowerShell while the container itself is Linux-based.

Byte 4: Docker Engine Runs Inside Linux**Builds on:**

Byte 3

In plain terms:

The **Docker Engine** is the actual system that creates, starts, stops, and manages containers.

The code:

docker info

Important output from our setup includes:

Context: desktop-linux

Kernel Version: ...-microsoft-standard-WSL2

Operating System: Docker Desktop

OSType: linux

What's happening:

OSType: linux confirms that Docker Engine is running in a Linux environment.

The WSL 2 kernel provides the Linux environment used by Docker Desktop.

Why it matters:

Even though our host computer is Windows, our Docker container is a **Linux container**.

Byte 5: Docker Compose Defines Our Single Container

Builds on:

Byte 4

In plain terms:

Docker Compose is used to define and manage our Docker application.

Previously, the project had four Compose services:

frontend

backend

postgres

nginx

We changed the architecture to **one service**:

app

The code:

services:

app:

What's happening:

Docker Compose now creates only one project container.

That single container contains:

- Vue frontend
- ASP.NET Core API
- PostgreSQL

- NGINX

Why it matters:

The complete Placement Tracker can now be started as **one Docker container**.

Byte 6: The Single Container Contains the Whole Application

Builds on:

Byte 5

In plain terms:

Instead of having four separate containers, we created a **single multi-stage Docker image**.

The Dockerfile performs different build stages.

Your Windows machine runs **Docker Desktop**, which uses **WSL 2 to provide the Linux environment** for the Docker Engine. Docker Compose then creates four Linux containers: Vue, ASP.NET Core, PostgreSQL, and NGINX. The containers communicate through Docker's internal network, while NGINX exposes the application on port 8080. You control all of this from Windows using commands such as `docker compose up`, `docker compose ps`, and `docker compose down`.

Byte 7: How the Single Container Starts

Builds on:

Byte 6

In plain terms:

A container needs a startup process that starts the required components.

We created:

`docker-entrypoint.sh`

This script controls the startup order.

Conceptually:

Container starts

↓

Start/initialize PostgreSQL

↓

Wait for PostgreSQL

↓

Start ASP.NET Core API

↓

Start NGINX

What's happening:

The entrypoint script ensures that PostgreSQL is available before the API starts.

Then NGINX starts and becomes the main entry point for the application.

Why it matters:

All required components start automatically when the single container starts.

Byte 8: How Components Communicate Inside One Container

Builds on:

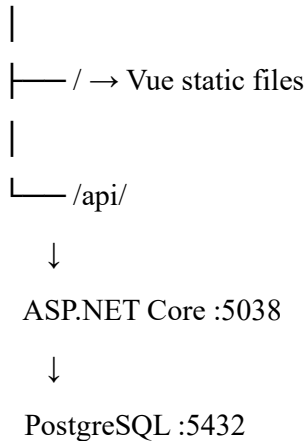
Byte 7

Previously, the containers communicated using Docker's internal network.

Now everything is inside the **same container**, so communication happens through the container's local network interface.

The architecture is:

NGINX :8080



Notice that the backend uses:

127.0.0.1:5432

for PostgreSQL.

And NGINX forwards API requests to:

127.0.0.1:5038

Why it matters:

Since all components are inside the same container, they can communicate through 127.0.0.1.

Byte 9: Why NGINX Uses Port 8080

Builds on:

Byte 8

In plain terms:

NGINX is the **public entry point** of our container.

The Docker port mapping is:

ports:

- "8080:8080"

This means:

Windows

localhost:8080

↓

Container :8080

↓

NGINX

NGINX then handles:

/ → Vue application

/api/ → ASP.NET Core API

So we open:

http://localhost:8080

Why it matters:

Only port **8080** needs to be exposed to the Windows host for the application.

Byte 10: Why ASP.NET Core Uses Port 5038 Internally

Builds on:

Byte 9

In plain terms:

NGINX already occupies port 8080 inside the container.

Therefore, ASP.NET Core cannot also listen on 8080.

So the API runs internally on:

127.0.0.1:5038

NGINX forwards:

/api/

↓

127.0.0.1:5038

↓

ASP.NET Core

The user still accesses everything through:

http://localhost:8080

Why it matters:

Two processes cannot listen on the same port, so NGINX uses 8080 and the API uses 5038.

Byte 11: PostgreSQL Data Persistence

Builds on:

Byte 7

In plain terms:

The PostgreSQL database is inside the same container, but its data is stored in a **Docker volume**.

The volume is:

pgdata

Conceptually:

PostgreSQL

↓

/var/lib/postgresql/data

↓

Docker volume

↓

pgdata

What's happening:

Even if the container is stopped and recreated, the database data can remain in the persistent volume.

Why it matters:

The database isn't dependent only on the temporary container filesystem.

Byte 12: Docker Compose Commands

Builds on:

All previous bytes

These are the main commands for our final setup.

Build and start

```
docker compose up --build -d
```

This:

1. Builds the Docker image.
2. Creates the single container.
3. Starts it in detached mode.

Check the container

```
docker compose ps
```

Expected result:

app

Only **one project container** should be running.

View logs

```
docker compose logs
```

Useful for checking:

- PostgreSQL startup
- ASP.NET Core startup
- NGINX startup
- errors

Stop the application

```
docker compose down
```

This stops and removes the Compose container while normally keeping the named database volume.

Byte 13: Docker Does Not Mean Ubuntu Must Be Running

Builds on:

Byte 3

Your computer has:

docker-desktop → Running → WSL 2

Ubuntu → Stopped → WSL 2

This is completely fine.

Docker Desktop uses:

docker-desktop

for its Docker Engine.

It does **not** require your separate Ubuntu distribution to be running.

If sir asks:

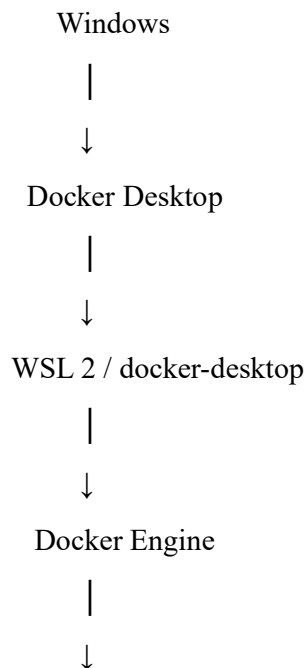
"Did you use Ubuntu for Docker?"

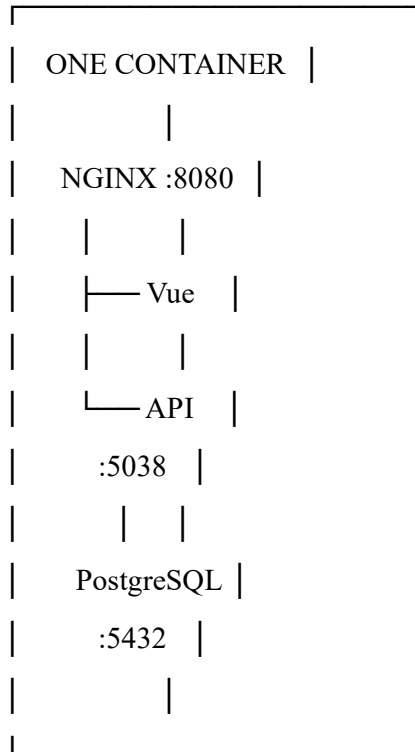
Answer:

"No. I used Docker Desktop with its WSL 2 backend. Docker Desktop runs the Docker Engine inside its own docker-desktop WSL 2 distribution. My separate Ubuntu WSL distribution isn't hosting the containers."

Byte 14: Final Architecture

Everything comes together like this:





Docker Volume
pgdata

The user accesses:

<http://localhost:8080>